

|         |                 |
|---------|-----------------|
| NAME-   | Prabhu Dayal    |
| SAP ID- | 590033396       |
| BATCH-  | 16 B.Tech C.S.E |

## Programming In C-HDOC DAY\_18

### Problem

Write a menu-driven C program using **switch-case** to perform digit-based operations on a positive integer. Define **one separate user-defined function for each menu option**, *with void return type and with a simple function call (without call by value/reference)*. Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

- Check whether it is a Strong number.
- Check whether it is an Armstrong number.
- Check whether it is a Harshad number.
- Check whether it is a Duck number.

### ALGORITHM

1. Start.
2. Input n.
3. Store n in copy.
4. Set temp = 0.
5. Repeat while copy > 0:
  - Find the last digit: ldig = copy % 10.
  - Set mul = 1.
  - Calculate the factorial of ldig.
  - Add the factorial to temp.
  - Remove the last digit: copy = copy / 10.
6. Compare temp with n.
7. If temp == n, print **Strong number**.
8. Otherwise, print **Not a Strong number**.
9. Stop.

---

### 2. Algorithm for Armstrong Number

**Definition:** A number is Armstrong if the sum of each digit raised to the power of the number of digits equals the original number.

**Algorithm:**

1. Start.
2. Input n.
3. Find the number of digits:
  - $\text{len} = \log_{10}(n) + 1$ .
4. Store n in copy.
5. Set sum = 0.
6. Repeat while copy > 0:
  - Find the last digit:  $\text{ldig} = \text{copy} \% 10$ .
  - Calculate  $\text{ldig}^{\text{len}}$ .
  - Add it to sum.
  - Remove the last digit:  $\text{copy} = \text{copy} / 10$ .
7. Compare sum with n.
8. If  $\text{sum} == n$ , print **Armstrong number**.
9. Otherwise, print **Not an Armstrong number**.
10. Stop.

For example, 153:

$$1^3 + 5^3 + 3^3 = 153$$

---

### 3. Algorithm for Harshad Number

**Definition:** A number is Harshad if it is divisible by the sum of its digits.

**Algorithm:**

1. Start.
  2. Input n.
  3. Store n in copy.
  4. Set sum = 0.
  5. Repeat while copy > 0:
    - Find the last digit:  $\text{ldig} = \text{copy} \% 10$ .
    - Add ldig to sum.
    - Remove the last digit:  $\text{copy} = \text{copy} / 10$ .
  6. Check whether  $n \% \text{sum} == 0$ .
  7. If true, print **Harshad number**.
  8. Otherwise, print **Not a Harshad number**.
  9. Stop.
-

#### 4. Algorithm for Duck Number

**Definition:** A Duck number contains at least one 0, but its first digit must not be 0.

**Algorithm:**

1. Start.
2. Input n.
3. Store n in copy.
4. Find the number of digits using  $\log_{10}(n)$ .
5. Find the first digit using:
  - $\text{istdig} = \text{copy} / 10^{\text{len}}$ .
6. If  $\text{istdig} == 0$ :
  - Print **Not a Duck number**.
7. Otherwise:
  - Set  $\text{index} = 0$ .
  - Repeat while  $\text{copy} > 0$ :
    - Find the last digit.
    - If the digit is 0, increment index.
    - Remove the last digit.
8. If  $\text{index} > 0$ , print **Duck number**.
9. Otherwise, print **Not a Duck number**.
10. Stop.

For example, 1024:

1024 → contains 0 → Duck number

---

#### 5. Main Program Algorithm

1. Start.
2. Declare num and c.
3. Input the positive integer num.
4. Display the choices:
  - Choice 1 → Strong number
  - Choice 2 → Armstrong number
  - Choice 3 → Harshad number
  - Choice 4 → Duck number
5. Input choice c.
6. Use switch(c):
  - If  $c = 1$ , call strong(num).

## HDOC DAY-18 REPORT

- If c = 2, call armstrong(num).
  - If c = 3, call harshad(num).
  - If c = 4, call duck(num).
  - Otherwise, print **Invalid choice**.
7. Stop.

## C PROGRAM CODE

```
#include <stdio.h>

#include <math.h>

void strong(int n)
{ int temp=0;int copy=n;
  while(copy>0)
  {
    int mul=1;
    int ldig=copy%10;
    for(int i=1;i<=ldig;i++)
    {
      mul=mul*i;
    }
    temp=temp+mul;
    copy=copy/10;
  }
  if(temp==n)
    printf("entered number is strong\n");
}

void armstrong(int n)
{ int len=((int)log10(n)+1);
  int copy=n;
  int sum=0;
  while(copy>0)
  {
    int ldig=copy%10;
    sum = sum + (int)(pow(ldig, len) + 0.5);
    copy=copy/10;
  }
```

## HDOC DAY-18 REPORT

```
}
printf("len = %d\n", len);
printf("sum = %d\n", sum);
if(sum == n)
    printf("the entered num is armstrong\n");
else
    printf("the entered num is not armstrong\n");
}

void harshad(int n)
{
    int copy=n;
    int sum=0;
    while(copy>0)
    {
        int ldig=copy%10;
        sum=sum+ldig;
        copy=copy/10;
    }
    if(n%sum==0)
        printf("the entered num is harshad no\n");
    else
        printf("the entered number is not harshad no\n");
}

void duck(int n)
{
    int copy=n;int index=0;
    int len=(int)log10(n);
    int istdig=copy/((int)pow(10,len));
    if(istdig==0)
    {
        printf("not a duck number\n");

    }
    else
    {
```

## HDOC DAY-18 REPORT

```
while(copy>0)
{
    int ldig=copy%10;
    if(ldig==0)
        index++;
    copy=copy/10;
}
if(index>0)
    printf("entered number is duck no");
else
    printf("the num is not duck");
}
}
int main(void)
{
    int num,c;
    printf("enter a positive integer\n");
    scanf("%d", &num);
    printf("choice 1 is to check wether the entered num is a strong no\n");
    printf("choice 2 is to check wether the entered num is a armstrong no\n");
    printf("choice 3 is to check wether the entered num is a harshad no\n");
    printf("choice 4 is to check wether the entered num is a duck no\n");
    printf("enter choice\n");
    scanf("%d", &c);
    switch(c)
    {
        case 1:
        {
            strong(num);
            break;
        }
        case 2:
        {
            armstrong(num);
            break;
        }
    }
```

## HDOC DAY-18 REPORT

```
}  
case 3:  
{  
    harshad(num);  
    break;  
}  
case 4:  
{  
    duck(num);  
    break;  
}  
default:{  
    printf("Invalid choice.\n");  
    break;  
}  
return 0;  
}  
}
```